

Progetto APS: Sistema di E-Voting per Referendum Universitario

WP1, WP2 & WP3: Architettura e Analisi di Sicurezza

Simone Donato, Alessio Ferrara

Maggio 2026

Indice

1	WP1: Definizione del Modello di Sistema e Analisi delle Minacce	2
1.1	Scenario Applicativo	2
1.2	Attori del Sistema	2
1.3	Funzionalità del Sistema	2
1.3.1	Requisiti Funzionali	2
1.3.2	Requisiti Non Funzionali	3
1.4	Threat Models (Modello di Minaccia)	3
1.5	Proprietà del Sistema da Preservare Sotto Attacco	3
2	WP2: Progettazione della Soluzione	5
2.1	Strategia di Generazione e Distribuzione delle Chiavi	5
2.2	Identificazione dello Studente e Fase di Registrazione (Blind Signature con FDH)	5
2.3	Formato dei Messaggi e della Scheda di Voto	6
2.3.1	Struttura Dati (JSON)	6
2.3.2	Operazione di Cifratura (Schema Ibrido AEAD)	7
2.4	Flusso di Comunicazione e Registrazione del Voto	7
2.5	Meccanismo di Sostituzione o Annullamento	8
2.6	Procedura di Scrutinio e Resilienza Operativa	9
2.7	Meccanismo di Verifica Individuale e Universale (Bulletin Board Autenticato)	10
3	WP3: Analisi di Sicurezza e Mitigazione delle Minacce	11
3.1	Obiettivo dell'Analisi	11
3.2	Analisi delle Proprietà Crittografiche	11
3.2.1	Anonimato e Unlinkability	11
3.2.2	Integrità e CCA-Security	11
3.2.3	Anti-Coercizione e Receipt-Freeness	11
3.3	Risoluzione dei Threat Models (TM)	12

1 WP1: Definizione del Modello di Sistema e Analisi delle Minacce

1.1 Scenario Applicativo

Il presente pacchetto di lavoro definisce l'architettura logica e i confini di sicurezza per un sistema di voto elettronico (E-Voting) centralizzato ma crittograficamente verificabile, configurato per consultazioni referendarie universitarie con dinamica di scelta binaria (*Si/No*, Scheda Bianca, Scheda Nulla). Il sistema si interfaccia con i servizi di Identity Provider dell'Ateneo per l'autenticazione, ma applica protocolli di disaccoppiamento crittografico per garantire che l'identità civile del votante sia matematicamente separata dalla preferenza espressa, preservando i principi di anonimato, unicità e non-coercibilità.

1.2 Attori del Sistema

In conformità con la scomposizione dei privilegi volta a eliminare singoli punti di fallimento (*Single Point of Failure*), si identificano i seguenti attori onesti che operano nel sistema:

- **Studente (Elettore):** Il titolare del diritto di voto. Interagisce con il sistema per autenticarsi, richiedere un'autorizzazione anonima, comporre la scheda cifrata e verificare l'inclusione della stessa. Ha il vincolo stringente di non poter generare prove di sottomissione a terzi (Anti-coercizione).
- **Server di Autenticazione (Ateneo):** Il gestore dell'identità. Autentica lo studente tramite credenziali istituzionali (SSO), verifica lo stato di avente diritto e rilascia un'autorizzazione crittografica cieca. Non deve mai entrare in contatto con la scheda di voto o con il gettone in chiaro.
- **Urna Digitale (Tallier):** Il collettore delle schede. Raccoglie i pacchetti cifrati inviati dagli studenti, ne valida l'autorizzazione formale a urne aperte e, a urne chiuse, esegue il conteggio previa ricostruzione della chiave di decifratura. Non conosce l'identità degli inseritori.
- **Commissione Elettorale (Trustees):** Un insieme di n entità garanti (docenti, personale, studenti) che detengono in modo frazionato la capacità di abilitare la decifratura dell'urna. Nessun membro può operare singolarmente.

1.3 Funzionalità del Sistema

1.3.1 Requisiti Funzionali

- **Accreditamento e Gestione Identità:** Autenticazione forte degli utenti tramite SSO istituzionale e verifica dello stato di iscrizione attivo per la consultazione corrente.
- **Emissione delle Credenziali Anonime:** Generazione di un'attestazione di legittimità al voto che non esponga l'identità dell'elettore.
- **Sottomissione Cifrata della Preferenza:** Raccolta di schede elettorali cifrate in modo end-to-end dal client dello studente fino all'Urna Digitale.

- **Scrutinio Algoritmico Condizionato:** Decifratura e computazione dei risultati aggregati attivabile esclusivamente al raggiungimento del quorum crittografico dei Trustees.

1.3.2 Requisiti Non Funzionali

- **Trasparenza e Verificabilità:** Ogni fase dello scrutinio deve essere pubblicamente verificabile da controllori esterni senza esporre i dati sensibili.
- **Efficienza Computazionale Lato Client:** Le operazioni demandate al dispositivo dello studente (generazione chiavi effimere, accecamento, cifratura) devono presentare complessità polinomiale ridotta, compatibile con smart wallet o dispositivi mobili.
- **Decentralizzazione del Controllo:** Lo sblocco dei dati dell'urna non è subordinato a un server centrale, ma ripartito su base distributiva tra i Trustees.

1.4 Threat Models (Modello di Minaccia)

Si formalizzano le minacce e i vettori di attacco considerati nell'analisi di sicurezza del sistema:

- TM.1 Iniezione e Falsificazione di Voti:** Un avversario esterno o uno studente malizioso tenta di inserire schede nell'urna senza autorizzazione o di duplicare voti legittimi (*Double Voting*).
- TM.2 De-anonimizzazione tramite Correlazione di Rete:** Un tecnico IT con accesso ai log di rete dei server di autenticazione e dell'Urna Digitale tenta di incrociare i timestamp delle richieste SSO con i timestamp di arrivo delle schede per associare la matricola alla preferenza.
- TM.3 Manomissione dell'Urna o della Commissione:** Un amministratore di sistema dell'Urna o una fazione minoritaria della Commissione tenta di alterare i record archiviati, eliminare voti validi o inserire preferenze arbitrarie prima o durante lo scrutinio.
- TM.4 Coercizione e Vendita del Voto:** Un attaccante offre un incentivo economico o applica una minaccia per costringere uno studente a votare una specifica preferenza, richiedendo una prova crittografica della scelta effettuata.
- TM.5 Interruzione e Crash dello Scrutinio:** Un attacco DoS o un guasto hardware durante la fase di tallying causa la perdita dei dati volatili o l'impossibilità di ricostruire lo stato del sistema.

1.5 Proprietà del Sistema da Preservare Sotto Attacco

L'architettura crittografica è progettata per garantire le seguenti proprietà di sicurezza stabili:

- **Anonimato (Confidenzialità):** Separazione matematica tra l'identità dello studente e la scheda depositata.

- **Integrità End-to-End:** Inalterabilità del voto dal momento della generazione sul client fino al calcolo matematico del tally.
- **Unicità del Voto:** Assicurazione che ogni gettone autorizzato pesi esattamente uno nel computo finale, scartando i tentativi di replay.
- **Receipt-Freeness:** Impossibilità per lo studente di produrre una ricevuta crittografica che dimostri a terzi la preferenza espressa, mitigando i tentati attacchi di coercizione.
- **Resilienza Operativa:** Capacità di completare lo scrutinio anche in caso di indisponibilità di una quota minoritaria di membri della Commissione.

2 WP2: Progettazione della Soluzione

2.1 Strategia di Generazione e Distribuzione delle Chiavi

La radice di fiducia e la separazione dei compiti si basano su una specifica configurazione di Infrastruttura a Chiave Pubblica (PKI) e schemi di suddivisione del segreto:

1. **PKI di Ateneo (Auth Server):** Il server di autenticazione genera una coppia di chiavi asimmetriche RSA stabili mediante l'algoritmo $Gen_{RSA}(1^\lambda)$. La chiave pubblica (e, n) viene pubblicata sulla directory d'Ateneo ed è cablata nel software client. La chiave privata d viene isolata all'interno di un modulo hardware sicuro (HSM) e utilizzata esclusivamente per apporre firme cieche.
2. **Chiave dell'Urna e Secret Sharing di Shamir:** L'Urna Digitale, in fase di setup dell'elezione, inizializza una coppia di chiavi asimmetriche (es. RSA o ECC). La chiave pubblica PK_{urna} viene distribuita pubblicamente per consentire la cifratura dei voti. La chiave privata SK_{urna} costituisce il segreto fondamentale per l'apertura dei voti; essa viene frammentata tramite uno **Schema di Shamir** (t, n) e distribuita agli n membri della Commissione Elettorale.

Formalmente, si definisce un polinomio casuale di grado $t - 1$ su un campo finito $\mathbb{Z}_p$:

$$f(x) = SK_{urna} + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1} \pmod{p}$$

Ogni coordinatore i della commissione riceve la quota segreta (x_i, y_i) dove $y_i = f(x_i)$. Al termine della distribuzione, la chiave originale SK_{urna} viene permanentemente sovrascritta e distrutta dalla memoria volatile dell'urna tramite istruzioni esplicite di *memory zeroization*. La soglia viene impostata a $t = \lceil 2/3 \cdot n \rceil$.

Nota sulle Assunzioni di Setup: Il presente modello assume l'Urna come *trusted* esclusivamente nella limitata fase iniziale di setup. In scenari ad altissima criticità, per eliminare il *Single Point of Failure* rappresentato dall'istante in cui l'Urna possiede in memoria la chiave SK_{urna} non ancora frammentata, è possibile ricorrere a protocolli di **Distributed Key Generation (DKG)**. In tale configurazione alternativa, sarebbero i membri della Commissione a generare congiuntamente le quote y_i e la PK_{urna} pubblica, senza che il segreto intero venga mai elaborato in chiaro su un singolo nodo.

2.2 Identificazione dello Studente e Fase di Registrazione (Blind Signature con FDH)

L'identificazione avviene tramite l'infrastruttura di Single Sign-On (SSO) dell'Ateneo. Per prevenire la falsificazione di credenziali basata sull'omomorfismo moltiplicativo intrinseco dell'algoritmo RSA (vulnerabilità del *Textbook RSA*), il sistema implementa lo schema **RSA-FDH (Full Domain Hash)** abbinato al protocollo di **Firma Cieca di Chaum**:

1. **Generazione e Blinding (Client):** Il dispositivo dello studente genera un gettone identificativo univoco $m \in \{0, 1\}^{128}$ tramite un CSPRNG. Prima dell'acceccamento, calcola l'hash crittografico del gettone $H(m)$ mappato sul dominio di RSA. Successivamente, seleziona un fattore di acceccamento casuale $r \in \mathbb{Z}_n^*$ tale che $\gcd(r, n) = 1$. Il client calcola il gettone accecato:

$$m' = H(m) \cdot r^e \pmod{n}$$

Il valore m' viene trasmesso al Server di Autenticazione unitamente al token di sessione SSO.

2. **Signing (Server):** Il Server verifica che la matricola sia presente nelle liste degli aventi diritto e che il flag `has_voted` sia falso. Se i controlli hanno esito positivo, il server imposta `has_voted = true` e calcola la firma cieca:

$$s' = (m')^d \pmod{n}$$

Il valore s' viene restituito al client.

3. **Unblinding (Client):** Lo studente riceve s' ed estrae la firma pulita moltiplicandola per l'inverso modulare del fattore di accecamento:

$$s = s' \cdot r^{-1} = (H(m) \cdot r^e)^d \cdot r^{-1} \equiv H(m)^d \cdot r \cdot r^{-1} \equiv H(m)^d \pmod{n}$$

La coppia (m, s) rappresenta la credenziale anonima di voto. La validità è verificabile da chiunque mediante la chiave pubblica dell'Ateneo controllando che $s^e \equiv H(m) \pmod{n}$. L'uso della funzione hash previene gli attacchi esistenziali basati sulla proprietà omomorfa del Textbook RSA.

2.3 Formato dei Messaggi e della Scheda di Voto

2.3.1 Struttura Dati (JSON)

La scheda S è formattata come oggetto JSON contenente i seguenti campi strutturati:

- **preferenza:** Stringa vincolata ai valori di dizionario ["SI", "NO", "BIANCA", "NULLA"].
- **gettone_m:** Stringa esadecimale rappresentante il gettone m a 128-bit.
- **firma_s:** Stringa esadecimale contenente la firma de-blinded s .
- **timestamp:** Marcatura temporale estesa in formato standard ISO 8601.
- **h_bind:** Hash di binding calcolato come:

$$\text{h_bind} = \text{SHA-256}(\text{preferenza} \parallel \text{gettone_m} \parallel \text{timestamp})$$

L'introduzione dell'hash di binding (**h_bind**) salda la scelta di voto al gettone e all'istante temporale, impedendo ad attori interni (es. gestori dell'urna) di modificare la preferenza alterando il payload JSON senza corrompere la validità dell'hash stesso.

```

{
  "preferenza": "SI" | "NO",
  "gettone_m": "valore_m",
  "firma_s": "valore_s",
  "timestamp": "2026-05-20T14:30:05Z"
}

```

Figura 1: Formato logico e serializzazione della scheda di voto (JSON)

2.3.2 Operazione di Cifratura (Schema Ibrido AEAD)

Per superare i limiti di dimensione del payload imposti da RSA-OAEP e garantire la resistenza contro attacchi al testo cifrato scelto (CCA-security), viene implementato uno schema di cifratura autenticata ibrida (KEM/DEM). Per evitare manipolazioni silenziose della chiave asimmetrica, il ciphertext della chiave viene incluso come *Associated Data* (AAD) nel calcolo del tag simmetrico:

1. Il client genera una chiave simmetrica effimera $k_{sym} \xleftarrow{\$} \{0,1\}^{256}$ e un Vettore di Inizializzazione crittograficamente sicuro $iv \xleftarrow{\$} \{0,1\}^{96}$. Poiché k_{sym} è generata ex-novo per ogni scheda, il rischio di riutilizzo del Nonce è strutturalmente nullo.
2. La chiave simmetrica viene cifrata con la chiave pubblica dell'Urna mediante RSA-OAEP, ottenendo l'incapsulamento della chiave:

$$C_{key} \leftarrow \text{RSA-OAEP}_{PK_{urna}}(k_{sym})$$

3. Il corpo del JSON della scheda S viene cifrato tramite algoritmo AES-256 in modalità GCM. Si utilizza C_{key} come dato associato (*AAD*) per legare indissolubilmente la parte simmetrica a quella asimmetrica:

$$(C_{sym}, \tau) \leftarrow \text{AES-GCM}_{k_{sym}, iv}(S, \text{AAD} = C_{key})$$

4. Il messaggio finale trasmesso all'urna è la stringa combinata: $C = (iv \parallel C_{key} \parallel C_{sym} \parallel \tau)$.

2.4 Flusso di Comunicazione e Registrazione del Voto

Il flusso operativo applica contromisure specifiche contro i vettori di attacco di analisi del traffico temporale (*Timing Attacks*):

1. **Preparazione ed Introduzione del Jitter:** Il client compone la scheda S , calcola la cifratura ibrida C . Per sventare l'analisi dei log incrociati da parte del *Tecnico IT* (TM.2), il client non invia il pacchetto immediatamente dopo la firma cieca, ma introduce un ritardo stocastico forzato (*jittering*) calcolato casualmente in un intervallo $\Delta t \in [1, 5]$ minuti, sufficiente a rompere l'allineamento statistico senza compromettere l'esperienza utente.

2. **Trasmissione Anonima:** Il pacchetto C viene trasmesso all'Urna Digitale intradando i pacchetti attraverso una rete anonimizzante (es. rete Tor o mix-net d'Ateneo) per mascherare l'indirizzo IP di origine.
3. **Batching e Registrazione sull'Urna:** L'Urna Digitale riceve i pacchetti e valida l'integrità strutturale del ciphertext simmetrico verificando il tag τ . I pacchetti validi non vengono memorizzati nell'ordine esatto di arrivo: l'urna gestisce code di accumulo orarie (*batching*), esegue un rimescolamento casuale (*shuffling*) dei blocchi ricevuti e solo successivamente esegue la persistenza sul database ad architettura *append-only*. L'urna rilascia come conferma l'hash $\text{SHA-256}(C)$.

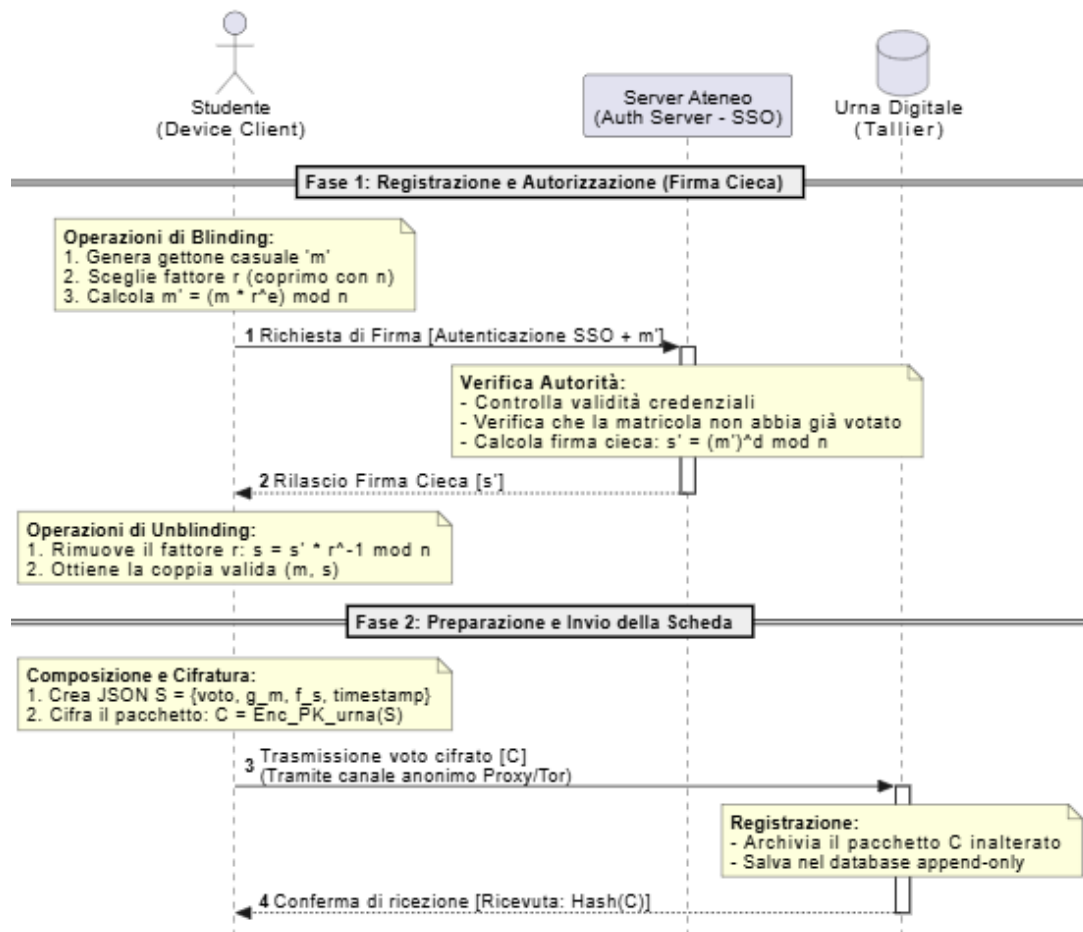


Figura 2: Diagramma di sequenza del flusso di autenticazione, blinding e sottomissione del voto

2.5 Meccanismo di Sostituzione o Annullamento

Per garantire l'assenza di ricevuta probante (*Receipt-Freeness*), il sistema supporta la sovrascrittura del voto:

- **Sostituzione:** Uno studente soggetto a coercizione può inviare una nuova scheda cifrata C_{new} riutilizzando lo stesso identico gettone m e la firma s , ma inserendo un valore di `timestamp` aggiornato.

- **Logica di Risoluzione:** L'Urna Digitale accetta e memorizza tutti i pacchetti in arrivo associati allo stesso gettone m . La logica di deduplicazione opererà in fase di scrutinio mantenendo esclusivamente il record associato al timestamp cronologicamente più recente, invalidando i precedenti.

2.6 Procedura di Scrutinio e Resilienza Operativa

Al termine della finestra temporale di votazione, l'Urna interrompe l'accettazione dei pacchetti di rete e avvia la fase di calcolo:

1. **Ricostruzione Idempotente del Segreto:** I membri della commissione si autenticano e presentano le proprie quote segrete (x_i, y_i) . Raggiunta la soglia di sicurezza t , il sistema esegue l'interpolazione di Lagrange per ricalcolare il polinomio nel punto zero:

$$SK_{urna} = f(0) = \sum_{i=1}^t y_i \prod_{j \neq i} \frac{-x_j}{x_i - x_j} \pmod{p}$$

Resilienza Operativa: Per mitigare crash hardware del server durante lo scrutinio (TM.5), il software di tallying implementa una logica di sottomissione idempotente. Le chiavi parziali caricate non modificano lo stato del database e, in caso di crash della memoria volatile, la procedura può essere rieseguita immediatamente reinserendo le medesime quote, senza richiedere una rigenerazione dei parametri.

2. **Decifratura, Filtraggio e Validazione:** L'Urna utilizza SK_{urna} per estrarre la chiave simmetrica k_{sym} da C_{key} , decifra il corpo della scheda mediante AES-GCM (utilizzando iv trasmesso in chiaro nel pacchetto C) e convalida i requisiti logici:
 - Verifica che la firma dell'Ateneo sia valida: l'Urna legge il campo `gettone_m` dalla scheda decifrata, ottiene m , calcola $H(m)$ e controlla che $s^e \equiv H(m) \pmod{n}$.
 - Verifica la congruenza dell'hash di binding: $h_bind \equiv \text{SHA-256}(\text{preferenza} \parallel \text{gettone_m} \parallel \text{timestamp})$.
 - Esegue la deduplicazione: se un gettone m compare in più schede decifrate, viene isolata solo la scheda con il timestamp più recente.
3. **Aggregazione:** Le preferenze delle schede convalidate vengono sommate nei registri dei risultati finali.

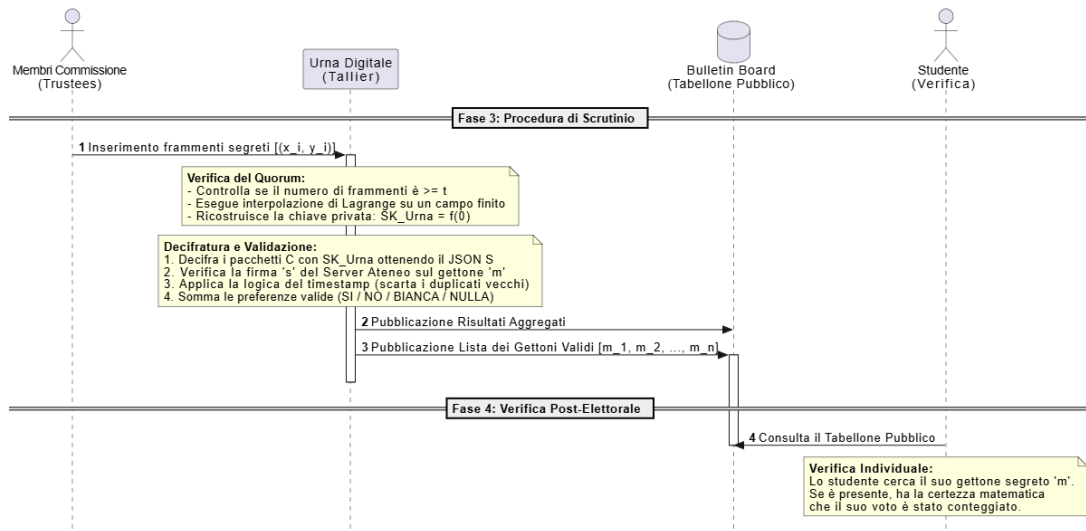


Figura 3: Diagramma di sequenza della fase di scrutinio, tallying e auditing post-elettorale

2.7 Meccanismo di Verifica Individuale e Universale (Bulletin Board Autenticato)

Al fine di garantire che l'ente gestore dell'urna non possa omettere voti legittimi o inserire voti fittizi senza essere rilevato, il sistema implementa un **Bulletin Board strutturato come Albero di Merkle (Merkle Tree)**:

- **Struttura del Tabellone:** I gettoni m_i estratti dalle sole schede convalidate e conteggiate vengono ordinati e posizionati come nodi foglia di un albero di hash binario. L'Urna calcola ricorsivamente gli hash intermedi fino ad ottenere una radice unica (*Merkle Root*). La Commissione Elettorale utilizza la chiave d'urna ricostruita per firmare digitalmente la Merkle Root, congelando la struttura dati in modo immutabile.
- **Verifica Individuale:** L'Urna pubblica il Merkle Tree e fornisce ad ogni client la relativa *Merkle Proof* (l'insieme dei nodi fratelli necessari per risalire dalle foglie alla root). Lo studente inserisce il proprio gettone segreto m nel modulo di verifica: il client calcola il percorso di hash e verifica la corrispondenza con la Root firmata dalla Commissione. Se l'hash coincide, vi è la certezza matematica che il voto è stato incluso nel computo.
- **Verifica Universale:** Qualsiasi entità esterna può scaricare l'intero Bulletin Board e verificare in modo indipendente l'elezione eseguendo tre controlli matematici:
 - (i) Verifica la firma digitale della Commissione sulla Merkle Root;
 - (ii) Verifica che ogni gettone m_i presente nelle foglie sia effettivamente valido, controllando la firma ad esso associata ($s_i^e \equiv H(m_i) \pmod{n}$);
 - (iii) Verifica l'assenza di duplicati foglia e ricalcola l'albero per controllare che la somma aritmetica dei voti corrisponda al risultato aggregato proclamato.

Nota sulla Privacy: La pubblicazione in chiaro dei gettoni m_i espone il fatto che un utente ha partecipato al voto, ma non rivela in alcun modo la preferenza espressa, mantenendo intatta la riservatezza della scheda.

3 WP3: Analisi di Sicurezza e Mitigazione delle Minacce

3.1 Obiettivo dell'Analisi

In questa sezione si dimostra formalmente la resilienza dell'architettura proposta rispetto alle minacce modellate nel WP1. L'analisi valuta come le primitive crittografiche e i protocolli di rete definiti nel WP2 garantiscano le proprietà di sicurezza richieste, operando sotto l'assunzione standard che gli algoritmi sottostanti (RSA, AES, SHA-256) siano computazionalmente sicuri.

3.2 Analisi delle Proprietà Crittografiche

3.2.1 Anonimato e Unlinkability

L'impossibilità di tracciare il legame tra identità civile ed espressione di voto è garantita dalla **Firma Cieca di Chaum con FDH**. Il Server di Ateneo firma il messaggio accecato $m' = H(m) \cdot r^e \pmod{n}$, senza poter mai osservare il gettone m in chiaro. Poiché il fattore r è scelto casualmente dallo studente in $\mathbb{Z}_n^*$, l'operazione di *unblinding* restituisce una coppia (m, s) matematicamente valida ma statisticamente indipendente da m' . Pertanto, anche nel caso peggiore in cui il Server di Ateneo e l'Urna Digitale dovessero colludere (condividendo i log di rete), non esisterebbe alcuna correlazione algebrica per risalire alla matricola partendo dal gettone.

3.2.2 Integrità e CCA-Security

L'integrità della singola scheda in transito è assicurata dal paradigma **AEAD (AES-GCM)**. La modalità *Galois/Counter Mode* calcola un tag di autenticazione τ sul testo cifrato, protetto dall'uso di un Nonce generato casualmente ad ogni trasmissione. Qualsiasi manipolazione anche di un singolo bit provocherà il fallimento della validazione del tag prima della decifrazione, proteggendo l'urna da attacchi attivi al testo cifrato scelto (*CCA-Security*) e attacchi di tipo *Padding Oracle*. L'integrità logica della preferenza è invece blindata dall'hash interno `h_bind`. Se un attaccante interno tentasse di alterare il campo `preferenza` in chiaro, il binding crittografico fallirebbe in fase di tallying:

$$\text{SHA-256}(\text{preferenza_alterata} \parallel \text{gettone} \parallel \text{timestamp}) \neq \text{h_bind}$$

3.2.3 Anti-Coercizione e Receipt-Freeness

Il protocollo è progettato per non fornire alcuna ricevuta che dimostri in modo inconfutabile la preferenza a un terzo. Se un coercitore obbligasse lo studente a votare in sua presenza, lo studente potrebbe, non appena al sicuro, generare un nuovo pacchetto C_{new} riutilizzando lo stesso gettone m , ma con un timestamp superiore e preferenza opposta. L'urna terrà valido solo il voto più recente.

Assunzioni critiche del modello: Questo meccanismo si regge su alcune assunzioni fondamentali. Primo, l'elettore deve avere accesso a un momento di privacy prima della chiusura delle urne. Secondo, **il gettone m deve restare rigorosamente segreto**: se il coercitore riuscisse a memorizzare o registrare il valore di m durante la sessione coercitiva, potrebbe in seguito consultare il Bulletin Board pubblico, compromettendo la *receipt-freeness* e verificando se il voto conteggiato per quel gettone è effettivamente quello

imposto. Infine, si assume l'impraticabilità su larga scala di attacchi di tipo *Midnight Voting*, in cui il coercitore impone il voto negli istanti immediatamente precedenti la chiusura delle urne, negando all'elettore la finestra temporale materiale necessaria per effettuare la sovrascrittura della propria preferenza.

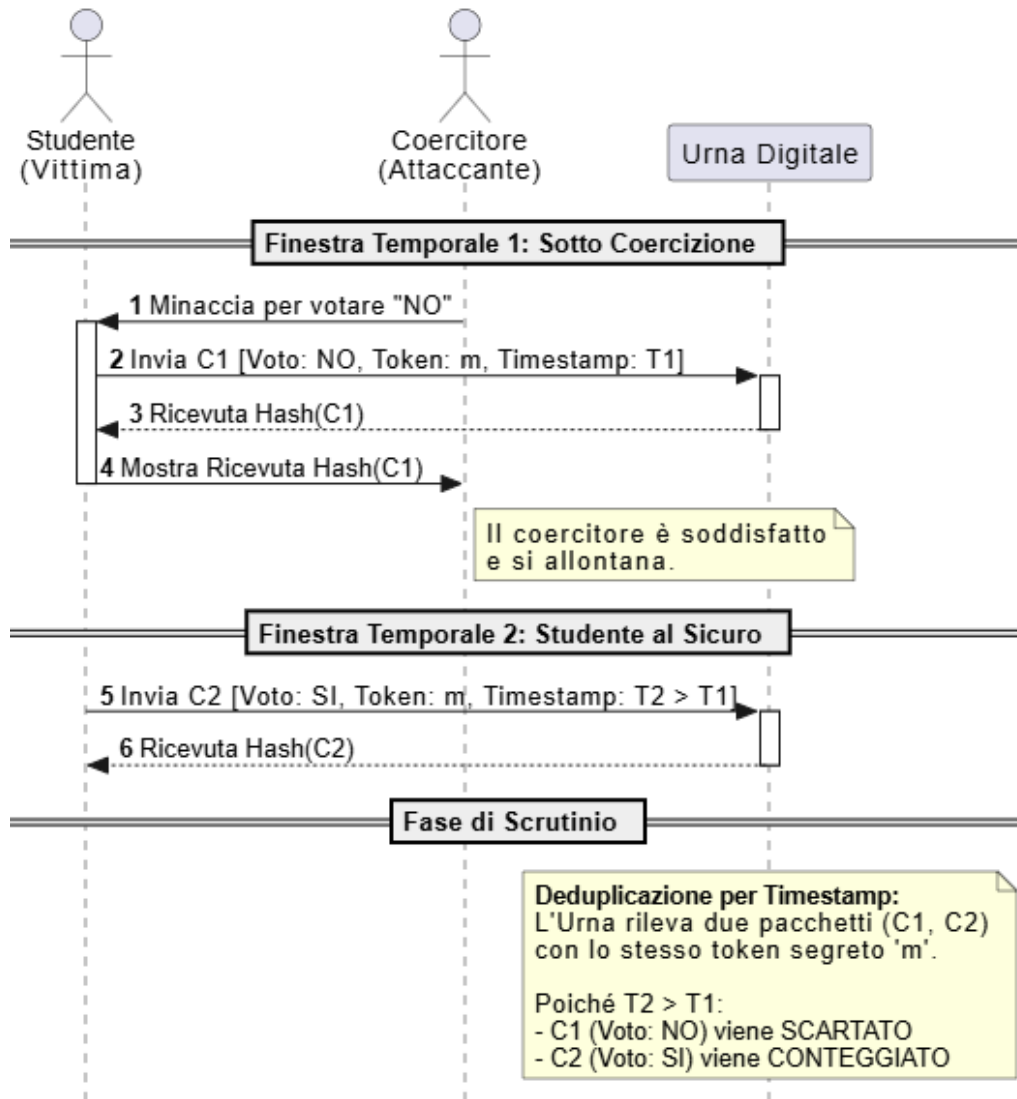


Figura 4: Meccanismo di sovrascrittura del voto per garantire la Receipt-Freeness.

3.3 Risoluzione dei Threat Models (TM)

Di seguito si mappa in modo analitico la mitigazione crittografica per ogni vettore di attacco identificato nel Modello di Minaccia (§1.4):

- **TM.1 - Iniezione e Falsificazione (Double Voting):** Forgiare un voto falso richiede di calcolare $s = H(m)^d \pmod{n}$ senza conoscere d , un'operazione resa intrattabile dalla durezza del problema RSA. L'uso dell'hashing (RSA-FDH) scongiura attacchi di tipo esistenziale o moltiplicativo che affliggono il Textbook RSA: non è possibile combinare firme valide su gettoni distinti per forgiarne una terza. I tentativi di *Replay Attack* vengono bloccati su due livelli distinti. A livello di rete, il controllo sull'hash $\text{SHA-256}(C)$ funge da prima barriera (ottimizzazione) per

scartare pacchetti identici. Tuttavia, **la vera garanzia crittografica contro il doppio voto risiede a livello logico (post-decifratura)**: è l'unicità del gettone m a forzare la deduplicazione sul tabellone. Anche se un attaccante costruisse un pacchetto completamente nuovo e crittograficamente valido contenente un gettone m già usato, il sistema conterebbe sempre e solo una singola preferenza per quell' m .

- **TM.2 - De-anonimizzazione tramite Timing Attack:** La minaccia del Tecnico IT viene neutralizzata dal disaccoppiamento temporale. L'impiego del **Jittering stocastico** ($\Delta t \in [1, 5]$ minuti) lato client distrugge l'allineamento deterministico tra i log SSO e il momento di invio in rete. A valle, il meccanismo di **Batching e Shuffling** dell'Urna disaccoppia ulteriormente l'orario di ricezione del pacchetto (livello di rete) dal suo indice di memorizzazione (livello di persistenza), rendendo inefficace l'analisi statistica del traffico.

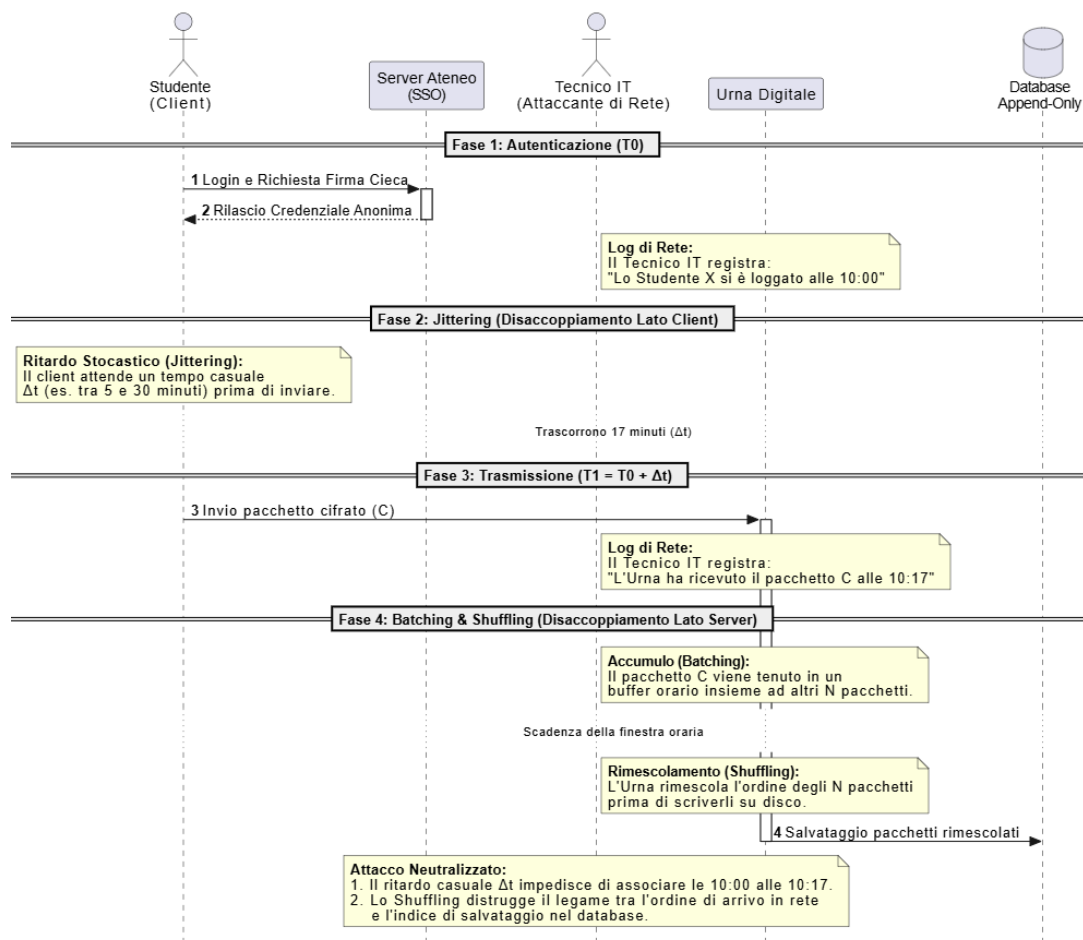


Figura 5: Disaccoppiamento temporale per la mitigazione dei Timing Attack.

- **TM.3 - Manomissione Urna o Commissione:** Una fazione minoritaria della Commissione non può forzare l'apertura anticipata dell'urna poiché il calcolo del polinomio di Lagrange richiede strettamente il raggiungimento della soglia t . Inoltre, l'Urna non può cancellare o omettere schede valide prima dello scrutinio grazie al **Bulletin Board autenticato**. L'omissione di un gettone m provocherebbe il fallimento immediato della *Merkle Proof* lato studente. *Modello di verifica:* Questa garanzia matematica è effettiva a patto che esista un sottoinsieme di elettori

motivati a eseguire la verifica individuale; è sufficiente che anche un solo studente denunci il fallimento della propria Merkle Proof per esporre in modo inconfutabile e crittografico la manomissione dell'Urna.

- **TM.4 - Coercizione e Vendita del Voto:** Come discusso nel paragrafo 3.2.3, la sovrascrittura incondizionata del voto in finestre temporali successive rende logicamente ed economicamente impraticabile l'acquisto della preferenza. Sotto la stretta assunzione che il gettone m non venga esposto visivamente al coercitore e dell'assenza di attacchi *Midnight Voting*, quest'ultimo non avrà mai alcuna garanzia che il voto espresso sotto minaccia non venga successivamente annullato o modificato dal legittimo titolare.
- **TM.5 - Interruzione e Crash dello Scrutinio:** L'eventualità di un guasto hardware, perdita di stato o attacco Denial of Service (DoS) durante la delicata unione dei segreti di Shamir è gestita dalla logica di **Idempotenza**. Poiché la ricostruzione di SK_{urna} e la verifica di τ avvengono come funzioni pure senza mutare irreversibilmente le quote fornite in input, il processo di scrutinio può essere ripetuto *ex-novo* al ripristino dell'infrastruttura, garantendo la totale continuità e resilienza operativa del referendum.